

Understanding Errors and Debugging

Day 4 Afternoon - Debugging, Exceptions, and Testing

30 min teaching + 30 min exercises

```
Traceback (most recent call last):  
  File "processor.py", line 18  
    percentage = completed / total  
ZeroDivisionError: division by zero
```

```
Observe → Read → Locate → Inspect  
      ↓  
Hypothesize → Change one thing →  
Run again
```

Goal: debug by reading evidence, not by guessing.

BLOCK 29

Where this block fits

The afternoon shifts from writing functions to proving and repairing behavior.



Instruction focus

- bug categories
- syntax errors
- runtime errors
- logic errors
- traceback reading
- systematic debugging

Practice focus

Students diagnose broken programs, record evidence, and fix one problem at a time.

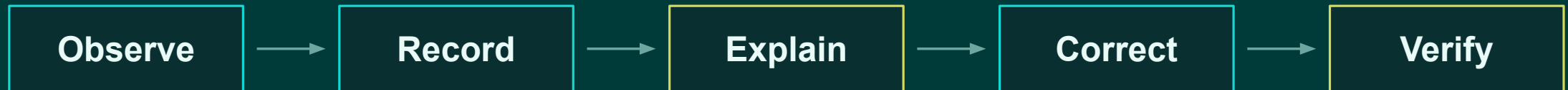
BLOCK 29

A bug is evidence, not failure

Professional debugging begins by observing what actually happened.

Beginner reaction
“It broke. I should change something.”

Debugger reaction
“What exactly failed, where, and with which values?”



Debugging is a process for turning symptoms into causes.

BLOCK 29

Three major error categories

Different kinds of bugs give different clues.

Syntax error

Python cannot understand the program structure.

```
if score >= 70  
    print("Pass")
```

Runtime error

The program starts, but fails while executing.

```
total = 10 / 0
```

Logic error

The program runs, but produces the wrong result.

```
area = length + width
```

Key idea

The first job is to identify which category you are dealing with.

BLOCK 29

Syntax error: Python cannot parse it

The code is not valid Python yet, so execution cannot begin.

```
score = 85  
  
if score >= 70  
    print("Pass")
```

```
if score >= 70:  
    print("Pass")
```

Clue

Python points near the place where the structure became invalid.

Common causes

Missing colon

Mismatched quotes

Bad indentation

Unclosed parentheses

Fix the structure first

BLOCK 29

Runtime error: valid code, bad execution

The code is legal Python, but something goes wrong while running.

```
completed = 10  
total = 0  
  
percentage = completed / total
```

Exception type
ZeroDivisionError

What it means
Python tried to divide by zero during execution.

```
record = {"record_id": "EQ-1001"}  
print(record["status"])
```

Another example
KeyError: the requested dictionary key is missing.

BLOCK 29

Logic error: no traceback, wrong answer

Python did exactly what the code said, but the code said the wrong thing.

```
length = 10  
width = 5  
  
area = length + width  
  
print(area)
```

Observed
output
15

Expected
output
50

Why harder?
There may be no error message. You need expected behavior to compare against actual behavior.

```
area = length * width
```

Logic errors require a testable expectation.

BLOCK 29

A traceback is a map

Do not read it as a wall of text. Read it for location and cause.

```
Traceback (most recent call last):  
  File "processor.py", line 18, in <module>  
    percentage = completed / total  
ZeroDivisionError: division by zero
```

Find these first

1. Exception type
2. Error message
3. Relevant file
4. Relevant line number
5. Code on that line

Bottom line

Line number

Failing operation

Relevant values

BLOCK 29

Read a traceback from the bottom up

The bottom usually tells you what happened.

```
ZeroDivisionError: division by zero
```

What happened?
Division by zero.

```
File "processor.py", line 18
```

Where?
processor.py, line 18.

```
percentage = completed / total
```

Which operation?
completed / total.

Next question
What was total at the moment this line ran?

BLOCK 29

Inspect the values near the failing line

Tracebacks show the line; debugging discovers why the values were there.

```
print(f"DEBUG completed={completed}")  
print(f"DEBUG total={total}")  
  
percentage = completed / total
```

Ask about each variable
Where was it created?
What type is it?
What value does it hold?
What values are allowed?

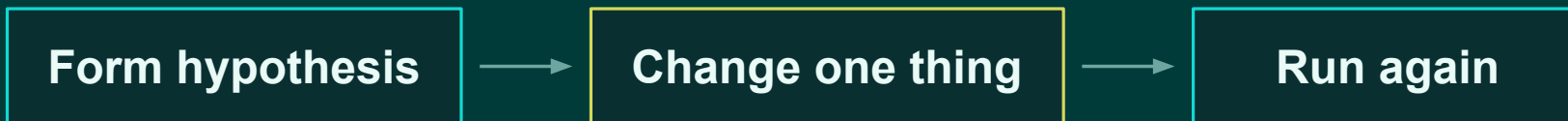
```
DEBUG completed=10  
DEBUG total=0
```

Evidence
The calculation failed because total was zero, not because division is broken.

BLOCK 29

Debugging has a repeatable loop

Use the same process instead of changing random lines.



Instructor phrase
A good debugger can explain why the change should fix the observed failure.

BLOCK 29

Debugging versus guessing

The difference is whether each change is based on evidence.

Guessing loop
Change random things
Run
Change more things
Run
Hope

Debugging loop
Read the failure
Locate the cause
Inspect values
Make one targeted change
Verify

Team habit
When asking for help, show the error message, the failing line, and the values you inspected.

BLOCK 29

Print debugging is useful

For beginners, simple inspection is often the fastest tool.

```
def calculate_percentage(completed, total):  
    print(f"DEBUG completed={completed}")  
    print(f"DEBUG total={total}")  
  
    return completed / total * 100
```

Use DEBUG labels

Make inspection output easy to distinguish from normal program output.

Remove or comment later

Debug prints are temporary learning tools. Later, logging is the professional version.

Print the value before the line that fails, not twenty lines later.

BLOCK 29

Isolate the problem

A smaller failure is easier to understand than a whole program failure.

Large program
100-line processor fails somewhere
inside the workflow.



Small reproduction
Call the failing function with the smallest
input that reproduces the problem.

```
print(calculate_percentage(10, 0))
```

```
record = {"record_id": "EQ-1001"}  
print(process_record(record))
```

Mental model

Shrink the mystery until the cause has nowhere to hide.

BLOCK 29

Common runtime clues

Exception names often describe the kind of problem.

ValueError

A value has the right type but invalid content.

Example: `int("twelve")`

TypeError

An operation received the wrong kind of object.

Example: `"5" + 10`

KeyError

A dictionary key is missing.

Example: `record["status"]`

ZeroDivisionError

The code attempted division by zero.

Example: `completed / total`

The exception type is the first clue, not the final answer.

BLOCK 29

Exercise toolkit: the bug report

Use the same worksheet for every broken program.

Record for each defect

Error category:

Observed behavior:

Relevant line:

Cause:

Correction:

Important rule

Fix one problem, run again, then
move to the next problem.

30-minute exercise block

Why this matters

Debugging notes help students separate symptoms from causes.

BLOCK 29

Exercise 1: syntax errors

Find the structural mistake, then make the smallest correction.

```
score = 82

if score >= 70
    print("Pass")
else:
print("Try again")
```

Task

Identify every syntax or indentation issue. Record the relevant line and correction.

5 minutes

Checkpoint

Does the program run before you start reasoning about the output?

BLOCK 29

Exercise 2: runtime errors

The code is valid Python, but bad values cause failures.

```
record = {  
    "record_id": "EQ-1001",  
    "completed": 18,  
    "total": 0,  
}  
  
percentage = record["completed"] /  
record["total"]  
print(record["status"])
```

Task

Run it. Read the traceback. Identify the first exception type and failing line.

6 minutes

Rule

Do not fix a later bug until the current traceback is understood.

BLOCK 29

Exercise 3: logic errors

No traceback means you need expected versus actual behavior.

```
completed = 18
total = 24

percentage = total / completed * 100

print(f"{percentage:.1f}%")
```

Expected
18 out of 24 should be 75.0%.

Task
Record actual output, cause, and correction. Then test with another value.

5 minutes

Logic errors are found by comparing reality to a known expectation.

BLOCK 29

Exercise 4: condition order defect

The program runs, but one branch prevents another from being reached.

```
error_count = 6

if error_count > 0:
    status = "warning"
elif error_count > 5:
    status = "rejected"
else:
    status = "ready"

print(status)
```

Task

Explain why 6 errors becomes warning instead of rejected.

Hint

More specific checks often need to come before broader checks.

5 minutes

BLOCK 29

Applied challenge: broken record processor

Find at least six defects across syntax, runtime, and logic errors.

```
records = [  
    {"record_id": " EQ-1 ", "file_name": "data.CSV",  
     "completed": 18, "total": 24, "errors": 0},  
    {"record_id": "", "file_name": "notes.txt",  
     "completed": 5, "total": 0, "errors": 6},  
]  
  
for record in records:  
    record_id = record["record_id"].strip().lower()  
    percentage = record["total"] / record["completed"] *  
100  
  
    if record["errors"] > 0:  
        status = "warning"  
    elif record["errors"] > 5:  
        status = "rejected"  
    else:  
        status = "ready"  
  
    print(record["status"] status)
```

Challenge task

Use the bug report format. Fix one defect at a time and rerun after each fix.

Look for

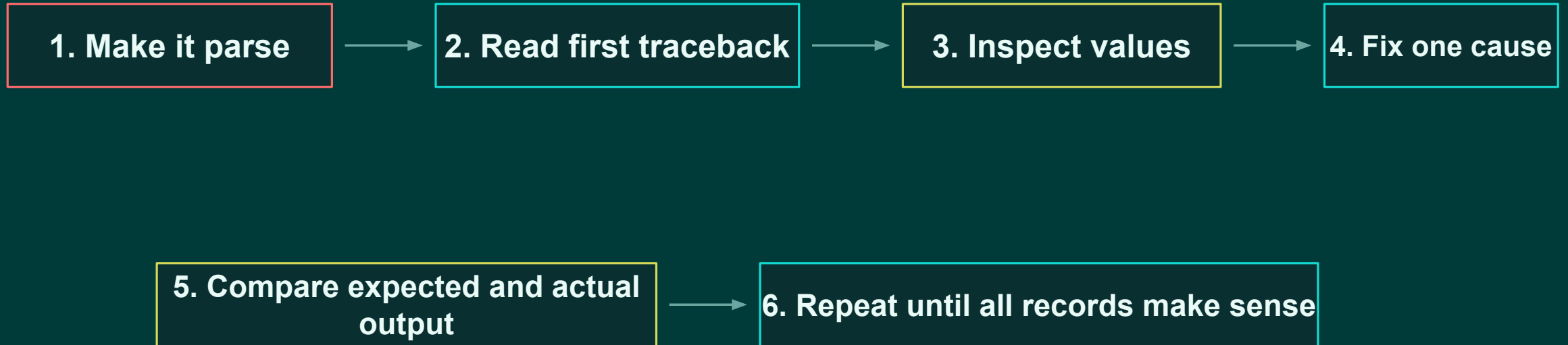
- syntax defect
- wrong dictionary key
- division issue
- wrong calculation
- bad condition order
- normalization issue

Remaining time

BLOCK 29

Debugging sequence for the challenge

Use this order to avoid getting lost.



Instructor reminder
Students should be able to explain every fix, not merely arrive at working code.

What good debugging notes look like

A correction is stronger when the cause is explicit.

Weak note

“Changed line 12 and it worked.”

Strong note

“Line 12 divided total by completed. The expected formula is completed / total.”

Example bug report

Category: logic error

Observed: 133.3% instead of 75.0%

Line: percentage calculation

Cause: numerator and denominator reversed

Correction: completed / total * 100

A good debugging note teaches the next person what happened.

BLOCK 29

Block 29 wrap-up

Students should leave with a repeatable process for diagnosing failures.

Students can identify

- syntax errors
- runtime errors
- logic errors
- exception types
- relevant traceback lines

Students can practice

- read the error
- inspect values
- isolate a problem
- make one change
- verify the fix

Big idea

Debugging is disciplined investigation: evidence first, changes second.

Next: handling expected failures with try, except, and intentional exceptions.